Certificate-Based Database Authentication

How and where mutual TLS between the backend and Postgres works in the code

Project: media-insight-engine
Scope: the containerized (Docker) path, where the server authenticates to Postgres with a client certificate instead of a password.

In one sentence

Instead of logging in with a password, the backend presents a client certificate over an encrypted TLS connection. Postgres checks that the certificate was signed by a trusted CA and that its name (CN=mie) matches the database role, and that IS the login. The certificate is handed to the backend at startup by HashiCorp Vault and lives only in RAM.

The four moving parts

- **cert-init (one-shot)** generates the CA, the server cert, and the client cert; seeds Vault.
- **Vault** the safe that stores the client cert + private key.
- **Postgres (db)** runs with TLS on and checks client certs instead of passwords.
- **Backend (server)** fetches its cert from Vault and connects with it.

The flow, step by step (with code)

1. **Certificates are created and stored**
   cert-init runs openssl to create a CA, a server cert, and a client cert whose CN is 'mie'. The client bundle is pushed into Vault; Postgres gets its server cert on a shared volume.

scripts/cert-init.sh:30-31 (client cert, CN=mie)

```
openssl req -new -nodes -newkey rsa:2048 \
  -keyout "$SRC/client.key" -out /tmp/client.csr -subj "/CN=mie"
openssl x509 -req -in /tmp/client.csr -CA "$SRC/ca.crt" ...
```

2. **Postgres comes up in TLS mode**
   docker-compose starts Postgres with ssl=on plus a server cert, key, and CA. All DB connections can now be encrypted, and the server can request a client cert.

docker-compose.yml:37-46

```
command:
  - postgres
  - -c
  - ssl=on
  - -c
  - ssl_cert_file=/etc/postgres/certs/server.crt
  - -c
  - ssl_key_file=/etc/postgres/certs/server.key
  - -c
  - ssl_ca_file=/etc/postgres/certs/ca.crt
```

3. **The auth rule says 'use the certificate, not a password'**
   pg_hba.conf is Postgres's rulebook. The hostssl line matches TLS connections to the mie database and authenticates them by client certificate (method 'cert'). clientcert=verify-full means the cert's CN must equal the role name. No password is involved on this path.

postgres/pg_hba.conf:8-9

```
# TYPE     DATABASE  USER  ADDRESS  METHOD
hostssl    mie       mie   all      cert clientcert=verify-full
host       mie       mie   all      scram-sha-256   # non-TLS fallback
```

4. **The backend is told to use cert auth, with no password in the URL**
   The server container runs with DB_AUTH=cert and a DATABASE_URL that has a username but NO password - the certificate is the credential.

docker-compose.yml:64-70

```
environment:
  DB_AUTH: cert
  DATABASE_URL: postgresql://mie@db:5432/mie   # no password
  VAULT_ADDR: http://vault:8200
  VAULT_TOKEN_FILE: /certs-shared/vault-token
  VAULT_DB_CERT_PATH: mie/db-client
  DB_CERT_DIR: /run/mie-certs
```

Note: /run/mie-certs is a tmpfs mount (docker-compose.yml:74-75), so the private key lives in RAM and never touches disk.

5. **At startup the backend fetches its cert from Vault**
   init_db() sees db_auth == 'cert' and calls secrets.ensure_db_certs(). That function reads the bundle from Vault over HTTP, writes ca.crt / client.crt / client.key into the tmpfs dir at 0600 (libpq rejects a world-readable key), and returns the libpq SSL parameters.

server/app/db.py:17-21 (the decision + the connect)

```
# Mutual-TLS cert auth: fetch the client cert from Vault and
# connect with sslmode=verify-full.
ssl = secrets.ensure_db_certs() if settings.db_auth == "cert" else {}
with psycopg.connect(settings.database_url,
                     autocommit=True, **ssl) as conn:
```

server/app/secrets.py:16-17, 54-60 (filename -> libpq param)

```
_CERT_FILES = {"ca.crt": "sslrootcert",
               "client.crt": "sslcert",
               "client.key": "sslkey"}
...
ssl = {"sslmode": "verify-full"}
for filename, param in _CERT_FILES.items():
    path = cert_dir / filename
    path.write_text(data[filename])
    path.chmod(0o600)        # libpq rejects world-readable key
    ssl[param] = str(path)
return ssl
```

6. **The TLS handshake authenticates both sides**
   psycopg/libpq connects with sslmode=verify-full and the three files. verify-full means the backend also verifies the SERVER's identity (correct CA + hostname 'db'), while it presents its own client cert. Postgres validates the client cert against the CA, reads CN=mie, maps it to the mie role, and authorizes the connection - all without a password.
7. **Every pooled connection reuses the same TLS settings**
   The same ssl kwargs are passed into the connection pool, so each connection the app borrows is a mutual-TLS connection.

server/app/db.py:26-28

```
_pool = ConnectionPool(settings.database_url,
    min_size=1, max_size=10, open=True,
    kwargs={"autocommit": True, "row_factory": dict_row, **ssl},
    configure=register_vector)
```

The one caveat to mention

Certificate auth only runs when DB_AUTH == 'cert', which is the Docker/production-style path. For local, non-Docker development there is a password fallback: the Makefile target connects with DB_AUTH=password and sslmode=disable, which deliberately matches the plaintext 'host ... scram-sha-256' rule in pg_hba.conf instead of the 'hostssl' cert rule.

Makefile:25 (local dev fallback)

```
DATABASE_URL="postgresql://mie:$$POSTGRES_PASSWORD@localhost:5437/mie?sslmode=disable"
DB_AUTH=password
```

Also worth noting: the Vault container runs in dev mode (in-memory, auto-unsealed, root token), so the certificate DELIVERY mechanism is dev-grade, even though the DB auth itself is real mutual TLS. Production hardening (persistent Vault, auto-unseal, cert rotation) is listed as future work in docs/superpowers/specs/2026-10-21-db-cert-auth-vault-design.md.

Where to look - file map

- **scripts/cert-init.sh** the openssl bootstrap that mints the CA, server, and client certs.
- **docker-compose.yml** wires up Vault, cert-init, Postgres (ssl=on), and the server (DB_AUTH=cert).
- **postgres/pg_hba.conf** the auth rule: hostssl ... cert clientcert=verify-full.
- **server/app/db.py** chooses cert vs password and makes the actual connection / pool.
- **server/app/secrets.py** fetches the cert bundle from Vault and returns the libpq SSL params.
- **docs/.../db-cert-auth-vault-design.md** the design rationale and the production-hardening TODO list.